

Министерство образования Республики Беларусь
Учреждение образования
**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ**

Факультет компьютерного проектирования
Кафедра инженерной психологии и эргономики
Дисциплина: Компьютерные системы и сети

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:

**ПРИЛОЖЕНИЕ, РЕАЛИЗУЮЩИЕ СЕТЕВУЮ ИГРУ
«КАРТОЧНЫЙ БЛЕФ»**

БГУИР КП 6-05-0612-01 003 ПЗ

Студент

Барбашёва А.А.

Руководитель

Болтак С.В.

Минск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Анализ предметной области.....	6
1.2 Обзор аналогов	6
1.3 Постановка задачи.....	8
2 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ.....	9
2.1 Выбор языка программирования.....	9
2.2 Библиотека создания графического интерфейса	9
2.3 Сетевые протоколы и сериализация.....	9
3 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРОГРАММНОГО ПРОДУКТА И ОПИСАНИЕ ПРОТОКОЛА СЕТЕВОГО ВЗАИМОДЕЙСТВИЯ	11
3.1 Проектирование архитектуры.....	11
3.2 Реализация на основе модели OSI.....	11
3.3 Описание протокола сетевого взаимодействия	12
4 ОПИСАНИЕ АЛГОРИТМОВ ИГРОВОГО ПРОЦЕССА	13
4.1 Алгоритм раздачи карт	13
4.2 Логика перехода хода	13
5 ОПИСАНИЕ РЕАЛИЗАЦИИ ГРАФИЧЕСКОГО ИНТЕРФЕЙСА.....	15
5.1 Организация пользовательского интерфейса.....	15
5.2 Реализация игрового стола и отрисовка графики.....	16
5.3 Обработка событий и взаимодействие с пользователем.....	16
6 ТЕСТИРОВАНИЕ СИСТЕМЫ	18
6.1 Функциональное тестирование.....	18
6.2 Проверка работы в сети.....	19
7 ИНСТРУКЦИЯ ПОЛЬЗОВАТЕЛЯ И ТРЕБОВАНИЯ К ПРОГРАММНО- АППАРАТНОМУ ОБЕСПЕЧЕНИЮ	21
7.1 Требования к программно-аппаратному обеспечению.....	21
7.2 Инструкция пользователя.....	21
7.2.1 Подключение к серверу и подготовка к игре.....	21
7.2.2 Игровой процесс и элементы управления	22
7.2.3 Механика проверки на блеф	23
ЗАКЛЮЧЕНИЕ	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26
ПРИЛОЖЕНИЕ А	27

ВВЕДЕНИЕ

В современном мире индустрия компьютерных развлечений стремительно развивается, и сегмент многопользовательских настольных и карточных игр занимает в ней значительную долю. Настольные игры исторически выполняли важную социальную функцию, объединяя людей за одним столом. С развитием интернета и сетевых технологий эта потребность не исчезла, а трансформировалась: пользователи хотят иметь возможность играть с друзьями или случайными соперниками из любой точки мира.

В связи с этим, разработка надежных и отказоустойчивых сетевых приложений, эмулирующих классические карточные механики, является весьма актуальной задачей. Одной из самых популярных карточных игр на социальное взаимодействие, психологию и интуицию является игра «Блеф» (также известная как «Верю — не верю»).

Перенос подобной механики в цифровую среду требует не только реализации правил самой игры, но и создания стабильной сетевой архитектуры [1], способной синхронизировать состояния между несколькими клиентами в реальном времени, предотвращать читерство и обеспечивать комфортный пользовательский опыт.

Целью данного курсового проекта станет разработка клиент-серверного приложения, реализующего сетевую игру «Карточный блеф». Приложение будет рассчитано на игру от 2 до 4 человек с использованием классической колоды из 52 карт.

Для достижения поставленной цели в ходе выполнения курсового проекта будут решены следующие основные задачи:

- Будет проведен анализ предметной области и детальная постановка задачи для клиентской и серверной частей;
- Будет спроектирована архитектура клиент-серверного взаимодействия;
- Разработается собственный кастомный протокол прикладного уровня на базе модели OSI с использованием JSON-сериализации;
- Будет реализована логика игрового процесса (включая механику "патронов" и проверку блефа), управление игровыми сессиями через систему очередей и многопоточная обработка подключений сервером;
- Будет создан интуитивно понятный пользовательский интерфейс с применением библиотеки Swing;
- Проведется комплексное тестирование разработанного программного продукта.

Разрабатываемое программное средство позволит автоматизировать процесс игры, возьмет на себя роль беспристрастного дилера, обеспечит честность (исключит подглядывание в чужие карты) и предоставит удобный функционал для взаимодействия игроков по сети. В качестве языка разработки будет использован Java, что обеспечит кроссплатформенность конечного продукта и возможность его запуска на различных операционных системах.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1 Анализ предметной области

Предметной областью данного проекта является сфера сетевых многопользовательских карточных игр. Игра «Блеф» построена на механике скрытой информации. Главное в ней — уметь убедительно обманывать соперников при сбросе карт и вовремя распознавать чужой блеф.

В классическом настольном варианте для игры требуется от 2 до 4 человек и стандартная колода из 52 карт. Правила заключаются в том, что карты раздаются поровну между всеми участниками.

В реализованной цифровой версии игры сервер жестко задает требуемый номинал карт для каждого хода, который последовательно возрастает по кругу (от Двоек до Тузов). Игроки по очереди выкладывают от 1 до 4 карт рубашкой вверх, подтверждая, что они соответствуют требуемому сервером номиналу. Следующий игрок имеет право либо сбросить свои карты, продолжив ход, либо усомниться в честности предыдущего игрока, вызвав проверку на блеф. Количество таких проверок ограничено механикой «патронов». В случае проверки карты вскрываются: если игрок солгал, он забирает всю стопку сброшенных на стол карт себе, а инициатору проверки возвращается патрон; если игрок был честен — всю стопку забирает тот, кто инициировал проверку, при этом его патрон сгорает. Победителем становится тот, кто первым избавится от всех карт на руках.

Перенос данной механики в формат компьютерной сети диктует строгие требования к архитектуре приложения. В цифровой среде роль судьи берет на себя централизованный узел — сервер. Именно сервер хранит эталонное состояние, скрывает данные о руках соперников и валидирует ходы, чтобы исключить читерство.

1.2 Обзор аналогов

На сегодняшний день на рынке существует множество платформ для игры в настольные и карточные игры в режиме онлайн (например, BoardGameArena, Tabletopia, Tabletop Simulator).

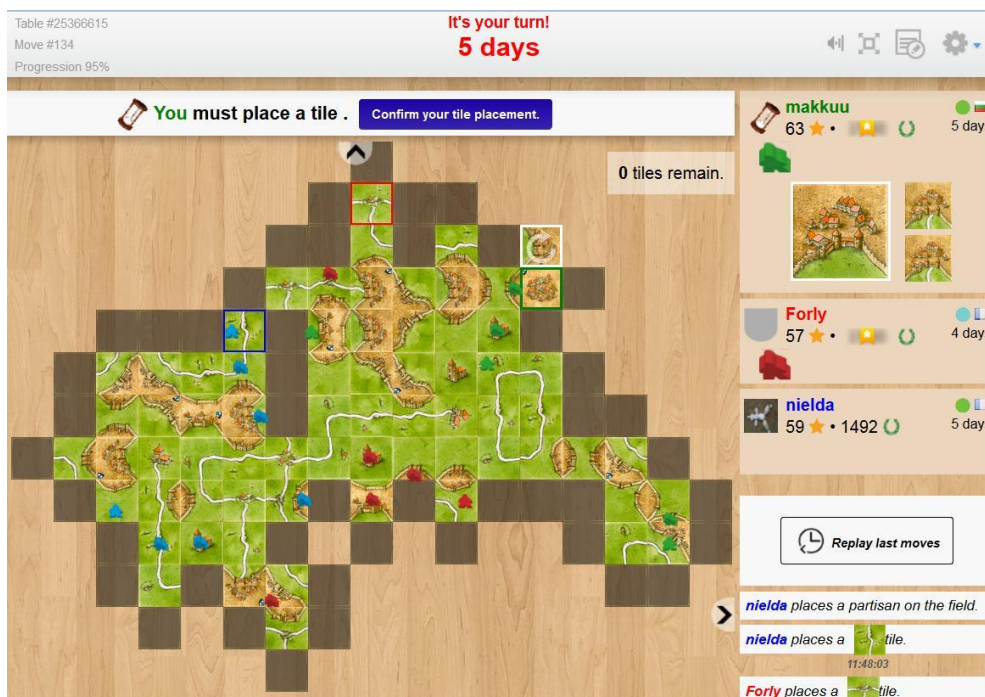


Рисунок 1.1 — Пример реализации карточных игр на платформе BoardGameArena

Большинство существующих решений являются либо масштабными физическими песочницами (Tabletop Simulator), которые требуют покупки и обладают высоким порогом вхождения, либо веб-сервисами, зависящими от работы браузера. Разрабатываемое же программное средство представляет собой легковесное, узконаправленное (standalone) desktop-приложение, которое фокусируется исключительно на механике одной игры. Это значительно снижает системные требования и упрощает процесс подключения игроков.

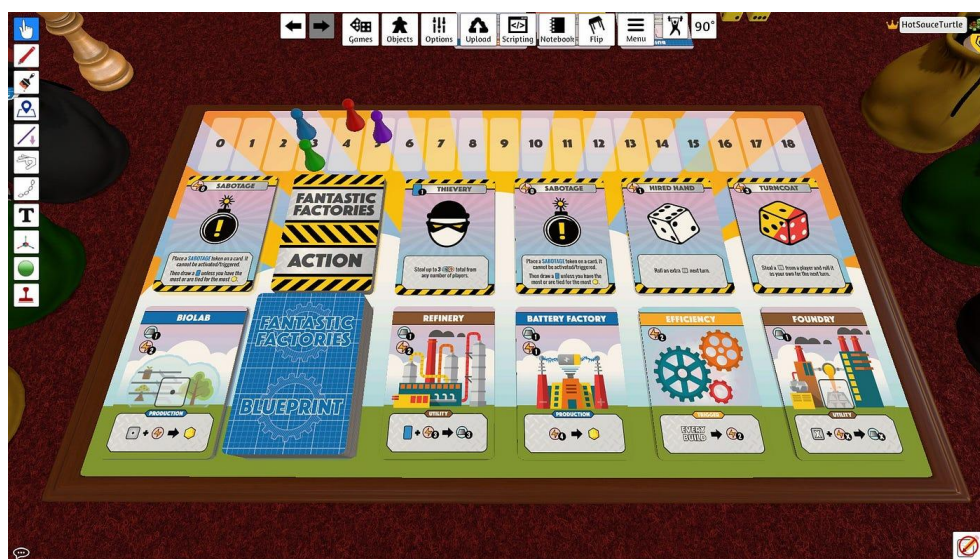


Рисунок 1.2 — Интерфейс Tabletop Simulator

1.3 Постановка задачи

В рамках данного проекта требуется разработать программный комплекс, состоящий из двух независимых модулей: «Сервер» и «Клиент».

Требования к серверной части:

- Ожидание и прием подключений от клиентов по протоколу TCP;
- Управление очередями матчмейкинга и поддержка одновременной игры от 2 до 4 игроков в одной сессии;
- Формирование и тасование колоды (52 карты);
- Реализация алгоритма раздачи карт, отслеживания очередности хода и автоматического повышения требуемого номинала карт;
- Проверка логики «верю / не верю» с учетом количества доступных «патронов» и последующим перераспределением карт;
- Сериализация данных для отправки клиентам (JSON).

Требования к клиентской части:

- Подключение к серверу по заданному IP-адресу и порту;
- Реализация пользовательского графического интерфейса с использованием библиотеки Swing;
- Отображение состояния игры: карты на руке пользователя, состояние стола, требуемый номинал карт и оставшиеся «патроны»;
- Возможность выбора карт для сброса;
- Возможность вызова действия «Кричать: БЛЕФ!»;
- Визуальное оповещение о победе или поражении в партии.

2 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

Успешность реализации сетевого программного средства напрямую зависит от правильно подобранного стека технологий. На этапе проектирования были проанализированы различные языки программирования, сетевые протоколы и методы сериализации данных.

2.1 Выбор языка программирования

В качестве основного языка программирования выбран Java с использованием набора средств разработчика JDK [2]. Основные преимущества Java для данного проекта:

- Кроссплатформенность: благодаря виртуальной машине Java (JVM) скомпилированный байт-код может выполняться на любой операционной системе;
- Многопоточность: Встроенные пакеты предоставляют безопасные инструменты для создания многопоточного сервера, способного обслуживать множество клиентов;

2.2 Библиотека создания графического интерфейса

Для реализации графического интерфейса пользователя (GUI) клиентской части применяется стандартная библиотека Java — Swing. Выбор Swing обоснован его наличием в стандартной библиотеке Java, отсутствием необходимости подключать внешние зависимости [3], а также легкостью кастомизации визуальных компонентов (отрисовка карт и игрового стола) с помощью переопределения метода `paintComponent`.

2.3 Сетевые протоколы и сериализация

Сетевое взаимодействие в проекте строится на основе модели взаимодействия открытых систем (OSI). На транспортном уровне используется протокол TCP [4] (Transmission Control Protocol) и абстракция сокетов (Sockets) [5]. В отличие от UDP, протокол TCP является протоколом с установлением логического соединения, гарантирующим целостность и сохранение порядка доставки пакетов. Карточная игра является пошаговой, и потеря пакета приведет к рассинхронизации всей партии, поэтому надежность TCP имеет критическое значение.

На уровне представления для обмена сообщениями между клиентом и сервером выбрана JSON-сериализация [6]. Формат JSON является легковесным и человекочитаемым, он обеспечивает удобство отладки

сетевого трафика и независимость от платформы. Для работы с JSON используется популярная библиотека Google Gson [7].

3 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРОГРАММНОГО ПРОДУКТА И ОПИСАНИЕ ПРОТОКОЛА СЕТЕВОГО ВЗАИМОДЕЙСТВИЯ

3.1 Проектирование архитектуры

Система строится по классической клиент-серверной архитектуре [8]. Это означает, что вся критически важная бизнес-логика выполняется на стороне сервера. Сервер представляет собой многопоточное приложение [9]. При запуске открывается `ServerSocket`. Для каждого нового клиента выделяется отдельный поток (обработчик `ClientHandler`), который отвечает за чтение и запись в сокет конкретного пользователя [10]. Логика самой игры вынесена в класс `GameSession`, который пересчитывает состояние игры и осуществляет широковещательную (broadcast) рассылку обновленного состояния всем клиентам, находящимся в текущей игровой сессии.

Клиент является приложением с графическим интерфейсом. Его главные задачи: отображение визуальной информации, захват действий пользователя (выделение карт мышью, нажатие кнопок «Сбросить карты», «Кричать: БЛЕФ!») и трансляция этих действий в JSON-сообщения, отправляемые на сервер.

3.2 Реализация на основе модели OSI

Сетевое взаимодействие приложения строго структурировано и проецируется на уровни модели OSI [11]:

- Прикладной уровень (Кастомный протокол): Реализованы специфические команды игры (`CONNECT`, `PLAYER_READY`, `PLAY_TURN`, `CALL_BLUFF`, `LOBBY_UPDATE`, `CARDS_DEALT`, `TABLE_UPDATE`, `YOUR_TURN`, `BULLETS_UPDATE`, `GAME_OVER`). Протокол описывает, как именно приложение реагирует на игровые события.

- Уровень представления (JSON): Преобразование объектов Java (например, списков карт) в строку формата JSON с помощью библиотеки `Gson` перед отправкой в сеть, и обратная десериализация при получении пакета.

- Сеансовый уровень (Управление сессиями): В классе `GameServer` реализована система очередей (matchmaking). При сборе нужного количества игроков создается экземпляр `GameSession`, который объединяет игроков в замкнутую группу от 2 до 4 человек. Сессия хранит текущее состояние колоды на столе, требуемый номинал, очередь хода и список участников.

- Транспортный уровень: Классы `Socket` и `ServerSocket` пакета `java.net`, реализующие надежную дуплексную передачу данных по протоколу TCP.

3.3 Описание протокола сетевого взаимодействия

Для корректного общения клиента и сервера разработан единый формат сообщений (класс Message). Каждое сообщение сериализуется в JSON-объект и содержит два основных поля: type (тип сообщения) и payload (полезная нагрузка в виде строки).

Пример структуры сообщения от клиента на сервер (игрок делает ход, выкладывая карты):

```
{
  "type": "PLAY_TURN",
  "payload": "[{\"rank\":\"TWO\",\"suit\":\"HEARTS\"}]"
}
```

Пример структуры сообщения от сервера клиенту (обновление состояния стола, где в поле payload через точку с запятой передаются: количество карт на столе, последний заявленный номинал и следующий требуемый номинал):

```
{
  "type": "TABLE_UPDATE",
  "payload": "1;Двойки;Тройки"
}
```

Такой подход позволяет абстрагировать сетевую транспортировку от бизнес-логики: парсер на стороне приемника считывает значение поля type и передает объект payload в соответствующий обработчик внутри системы.

4 ОПИСАНИЕ АЛГОРИТМОВ ИГРОВОГО ПРОЦЕССА

4.1 Алгоритм раздачи карт

Алгоритм раздачи карт запускается на стороне сервера автоматически, когда все подключенные к игровой сессии участники подтверждают свою готовность через сетевую команду. За выполнение этого процесса отвечает метод `dealCards()`. Для генерации игровых элементов применяется вспомогательный класс `Deck`, моделирующий стандартную колоду из 52 карт. Перед началом распределения карт вызывается метод `shuffle()`, случайным образом перемешивающий элементы колоды для обеспечения непредсказуемости игрового процесса.

Далее рассчитывается количество карт, выдаваемых каждому участнику, по формуле $\text{cardsPerPlayer} = \text{deck.cardsLeft()} / \text{players.size()}$. Сервер в цикле извлекает карты из колоды с помощью метода `drawCard()` и передает их в локальный список карт (руку) каждого объекта `ClientHandler`. После завершения цикла сформированные списки сериализуются и рассылаются соответствующим клиентам через пакет `CARDS_DEALT`. Чтобы подготовить игровой стол к началу партии, сервер отправляет всем клиентам команду `TABLE_UPDATE`. Это сообщение дает команду клиентским интерфейсам визуально очистить центральную стопку сброса.

4.2 Логика перехода хода

Логика управления очередностью игровых действий инкапсулирована в методе `nextTurn()`. При старте сессии первый ходящий игрок выбирается случайным образом в методе `startFirstTurn()`, а начальным целевым номиналом устанавливается минимальный ранг (`Rank.TWO`). Сервер отправляет активному участнику сообщение `YOUR_TURN`, содержащее локализованное текстовое название номинала карт, которые необходимо выложить (например, «Двойки», «Тройки»).

Участник в свой ход обязан выбрать от 1 до 4 карт и отправить их серверу в пакете `PLAY_TURN`. Сервер в методе `handlePlayTurn()` проверяет, является ли отправитель текущим активным игроком. При успешной валидации карты удаляются из руки игрока и перемещаются в общую стопку на столе `tablePile`, а также дублируются в буфер последнего хода `lastPlayedCards`.

После этого целевой номинал автоматически сдвигается на следующую позицию по кругу с помощью метода `advanceTargetRank()`, реализующего циклическое смещение по перечислению рангов карт. Индекс активного игрока обновляется по формуле $\text{currentPlayerIndex} = (\text{currentPlayerIndex} + 1) \% \text{players.size()}$, после чего повторно вызывается метод `nextTurn()`, передающий право хода следующему участнику. Если после сброса карт рука игрока оказывается пустой, сессия завершается объявлением победителя и отправкой

команды GAME_OVER. Все изменения состояния стола транслируются игрокам через сообщения TABLE_UPDATE.

4.3 Алгоритм проверки условий «верю/не верю»

Метод handleCallBluff() отвечает за главную механику игры — проверку на блеф. Если игрок не хочет сбрасывать свои карты, он может обвинить предыдущего участника во лжи, отправив серверу команду CALL_BLUFF. При обработке данного запроса сервер выполняет следующие проверки:

- Проверяется наличие карт в центре стола: если буфер lastPlayedCards пуст, вызов проверки блокируется, а игроку возвращается право совершить обычный ход.

- Проверяется лимит доступных обвинений («патронов»): если у вызывающего игрока количество оставшихся попыток равно нулю, сервер отклоняет запрос.

Если предварительные условия соблюдены, у вызывающего игрока списывается одна единица лимита (caller.useBullet()), после чего запускается цикл анализа карт из буфера lastPlayedCards. Сервер сравнивает реальный номинал каждой сброшенной карты (c.rank()) с последним объявленным номиналом (lastDeclaredRank). На основе этого анализа возможны два сценария:

1. Блеф успешно вскрыт. Если хотя бы одна карта не соответствует заявленному номиналу, обвинение признается верным. Игрок, совершивший обман (lastPlayer), обязан забрать всю накопленную стопку tablePile в свою руку. Вызывающему игроку (caller) возвращается потраченный «патрон» (caller.addBullet()), а право следующего хода передается ему.

2. Обвинение ошибочно. Если все сброшенные карты строго соответствуют заявленному номиналу, ходивший игрок признается честным. В этом случае всю стопку карт со стола вынужден забрать сам вызывающий игрок (caller), его потраченный «патрон» окончательно сгорает, а право следующего хода переходит к честному игроку (lastPlayer).

По завершении любого из сценариев стопка карт на столе tablePile и буфер последнего хода lastPlayedCards полностью очищаются, текущий требуемый номинал сбрасывается до значения lastDeclaredRank, всем клиентам отправляется обновляющий пакет TABLE_UPDATE со значением "0;-;", и управление передается методу nextTurn().

5 ОПИСАНИЕ РЕАЛИЗАЦИИ ГРАФИЧЕСКОГО ИНТЕРФЕЙСА

5.1 Организация пользовательского интерфейса

Графический интерфейс написан на встроенной библиотеке Java Swing, что позволяет запускать игру на разных ОС без установки дополнительных модулей. В качестве главного окна приложения используется класс-наследник `JFrame`, который отвечает за отрисовку базового окна и обработку действий пользователя (например, закрытие программы).

Пользовательский интерфейс логически разделен на несколько независимых панелей (наследников `JPanel`), смена которых происходит в зависимости от текущего состояния сетевой сессии. Разработаны следующие основные экраны: – панель подключения к серверу и ожидания в лобби; – панель игрового стола.

Смена экранов осуществляется с использованием менеджера компоновки `CardLayout`, что позволяет динамически переключать отображаемые интерфейсы без необходимости пересоздания главного окна приложения. Внешний вид панели подключения к серверу и ожидания подключения других участников игры представлен на рисунке 5.1.

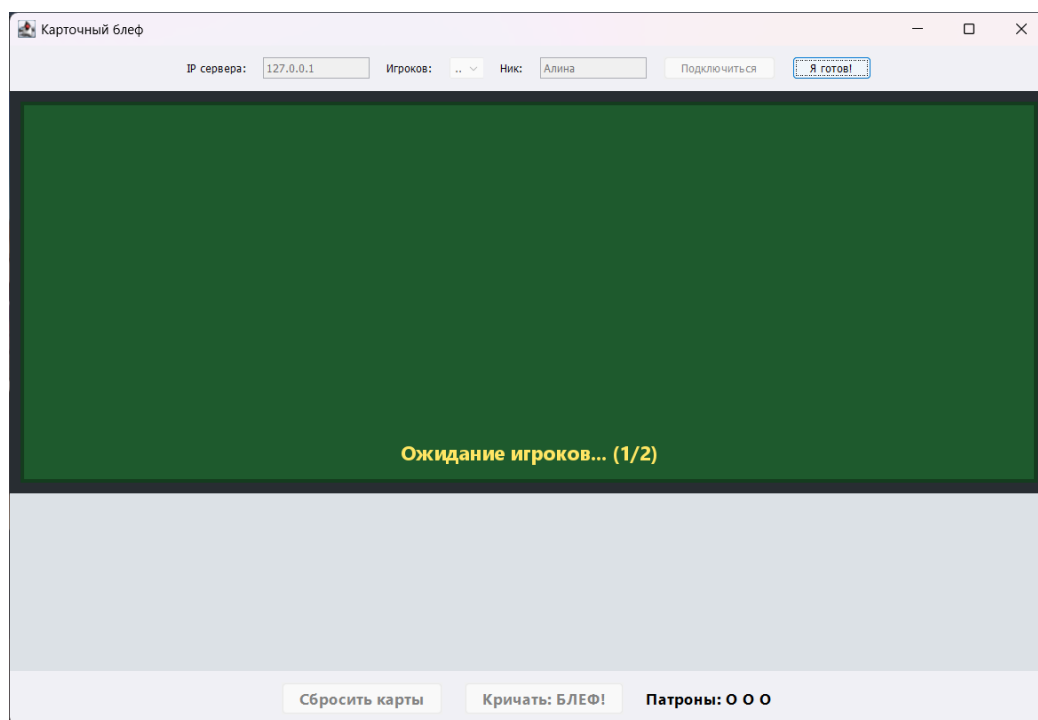


Рисунок 5.1 – Панель подключения к серверу и ожидания игроков

5.2 Реализация игрового стола и отрисовка графики

Ключевым элементом взаимодействия пользователя с игрой является панель игрового стола. Данная панель отвечает за визуализацию текущего состояния партии: отображение карт на руках игрока, центральной стопки сброса, информации о соперниках, требуемом номинале карт и оставшихся «патронах». Графический интерфейс активного игрового процесса представлен на рисунке 5.2.



Рисунок 5.2 – Интерфейс панели игрового стола в процессе партии

Отрисовка графических элементов реализована посредством переопределения метода `paintComponent(Graphics g)`. Карты представлены в виде масштабируемых растровых изображений, загружаемых в память при инициализации приложения. Для позиционирования карт на руках игрока применяется специальный алгоритм вычисления координат, обеспечивающий их равномерное распределение, центрирование и частичное перекрытие в виде веера в зависимости от текущего количества карт на руке.

5.3 Обработка событий и взаимодействие с пользователем

Взаимодействие пользователя с приложением реализовано на основе механизма слушателей событий (Event Listeners). Для обработки действий с визуальными картами применяется интерфейс `MouseListener`. При клике левой кнопкой мыши по области, соответствующей координатам конкретной карты,

она меняет свое визуальное состояние (сдвигается вверх по оси Y), что сигнализирует о ее выборе для последующего сброса.

Помимо взаимодействия с графическими объектами карт, интерфейс содержит набор управляющих элементов, реализованных с помощью компонента JButton. К основным управляющим воздействиям относятся: – подтверждение хода (отправка выделенных карт в стопку сброса); – вызов проверки на блеф («Кричать: БЛЕФ!»).

При активации данных элементов управления генерируются события ActionEvent, которые обрабатываются внутренними контроллерами. Контроллеры валидируют действия пользователя (например, проверяют, выбрано ли допустимое количество карт — от 1 до 4), формируют пакеты данных в формате JSON и передают их модулю сетевого взаимодействия для отправки на сервер по протоколу TCP. Визуальный отклик интерфейса на выбор пользователем карт для предстоящего хода представлен на рисунке 5.3.

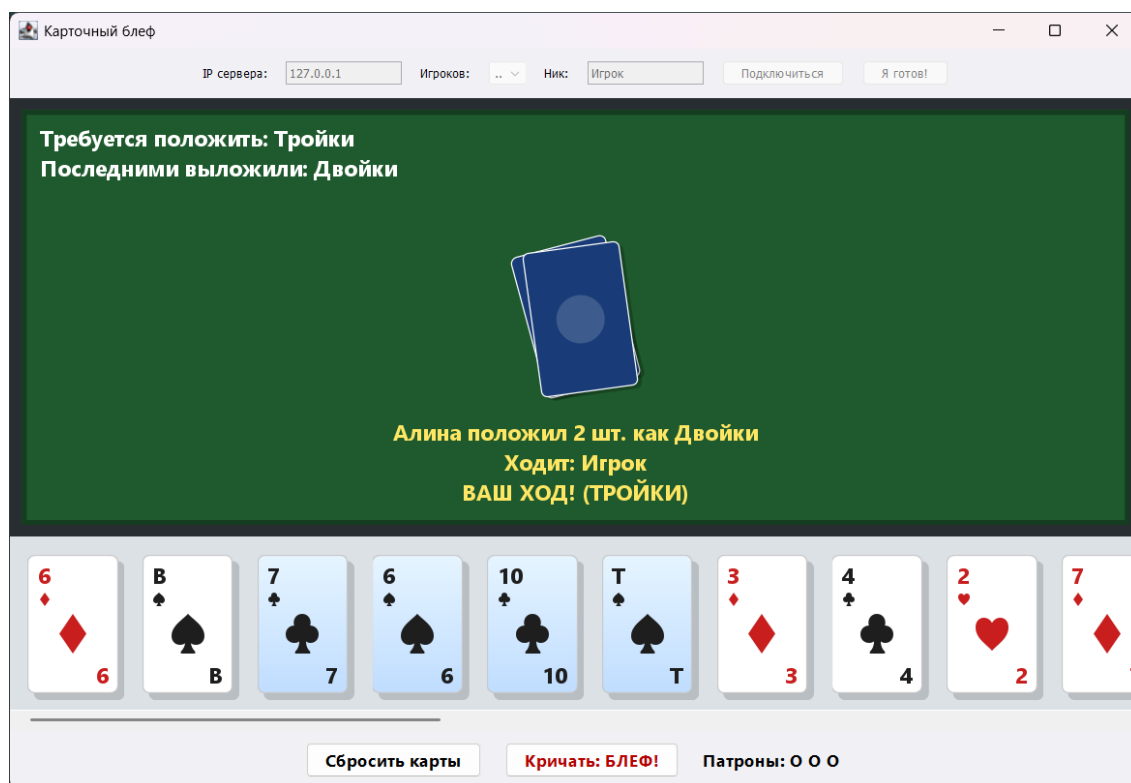


Рисунок 5.3 – Визуализация выбора карт пользователем перед совершением хода

Чтобы интерфейс игры не зависал при ожидании данных по сети, все визуальные обновления (например, реакция на пакеты TABLE_UPDATE или YOUR_TURN) выполняются в отдельном потоке отрисовки (Event Dispatch Thread) с помощью SwingUtilities.invokeLater() [12]. Данный подход гарантирует потокобезопасность библиотеки Swing и корректное визуальное оповещение игрока о любых изменениях на столе.

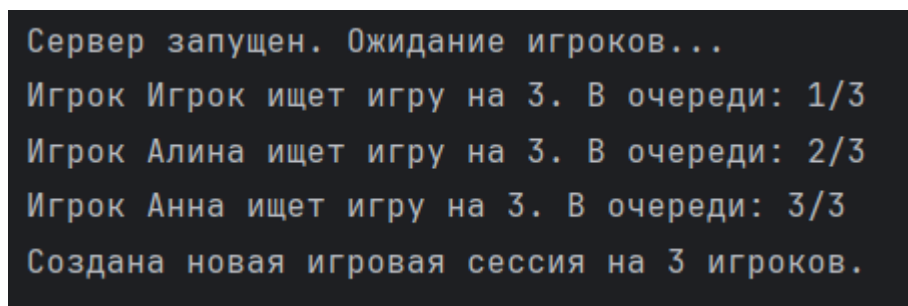
6 ТЕСТИРОВАНИЕ СИСТЕМЫ

В данном разделе описывается процесс проверки работоспособности разработанного клиент-серверного приложения «Карточный блеф». Процесс разделен на два этапа: функциональное тестирование игрового процесса и пользовательского интерфейса, а также проверка сетевого взаимодействия компонентов с использованием анализатора трафика Wireshark.

6.1 Функциональное тестирование

На этом этапе тестирования мы проверяем, правильно ли сервер реагирует на действия игроков и корректно ли завершается партия при победе одного из них.

Тестирование инициализации сервера и регистрации клиентов. В ходе данного теста производится запуск сервера и подключение к нему тестовой группы клиентов. Особое внимание уделяется проверке логики матчмейкинга: система должна позволять выбирать переменное количество игроков (вплоть до трех) и не стартовать игру преждевременно, если для сессии на троих подключилось только два пользователя. Подтверждением корректной работы служит вывод системной информации в консоль сервера, где фиксируются факты подключения пользователей в соответствующую очередь и изменения статуса игрового лобби. Данный результат представлен на рисунке 6.1.



```
Сервер запущен. Ожидание игроков...  
Игрок Игрок ищет игру на 3. В очереди: 1/3  
Игрок Алина ищет игру на 3. В очереди: 2/3  
Игрок Анна ищет игру на 3. В очереди: 3/3  
Создана новая игровая сессия на 3 игроков.
```

Рисунок 6.1 – Журнал логов сервера при успешном подключении клиентов

Тестирование логики завершения игры (Game Over). Тест проверяет алгоритм определения победителя и завершения игровой сессии. По правилам игры, участник, избавившийся от всех карт на руках, объявляется победителем. В рамках теста игровой процесс доводится до финальной стадии, где один из игроков делает успешный ход своей последней картой. Ожидается, что сервер зафиксирует окончание игры, разошлет сообщение с типом GAME_OVER всем клиентам, а интерфейс корректно отобразит финальный статус. Подтверждением отработки данного алгоритма является вывод на экраны клиентов сообщения о завершении игры с указанием победителя и

блокировка дальнейших игровых действий. Результат тестирования представлен на рисунке 6.2.

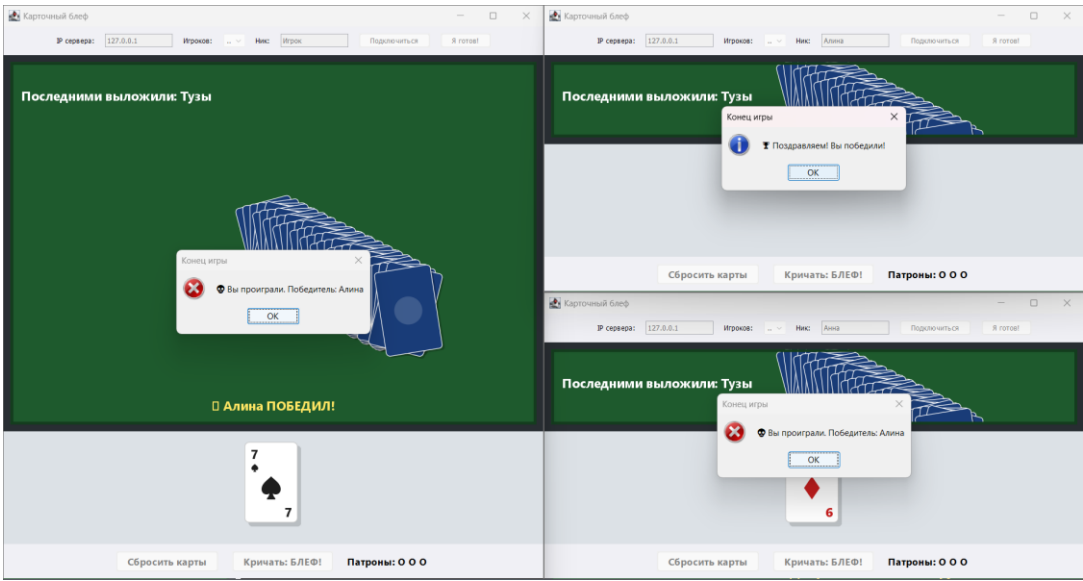


Рисунок 6.2 – Отображение результатов сессии при завершении игры

6.2 Проверка работы в сети

Для проверки корректности обмена данными по сети и анализа передаваемых пакетов использовался программный анализатор трафика Wireshark. Взаимодействие базируется на протоколе транспортного уровня TCP, а на уровне представления данные сериализуются в формат JSON.

Тестирование установки TCP-соединения. Этот тест проверяет, как клиент и сервер устанавливают связь. При запуске клиента происходит стандартное «тройное рукопожатие», гарантирующее установку надежного соединения на порту 8080. Подтверждением успешной установки соединения служит перехват соответствующих синхронизирующих пакетов (SYN, SYN-ACK, ACK) в анализаторе трафика. Процесс установки соединения зафиксирован на рисунке 6.3.

tcp.port == 8080					
Time	Source	Destination	Protocol	Length	Info
23 6.454420900	127.0.0.1	127.0.0.1	TCP	56	38397 → 8080 [SYN] Seq=0
24 6.454518800	127.0.0.1	127.0.0.1	TCP	56	8080 → 38397 [SYN, ACK]
25 6.454541600	127.0.0.1	127.0.0.1	TCP	44	38397 → 8080 [ACK] Seq=1

Рисунок 6.3 – Процесс установки TCP-соединения между клиентом и сервером

Тестирование обмена игровыми сообщениями. Данный тест проверяет корректность формирования, отправки и получения пользовательских JSON-сообщений. При выполнении игрового действия (например, сброс карт)

клиент формирует объект сообщения типа PLAY_TURN, сериализует его в строку JSON и отправляет на сервер. Ожидается, что структура передаваемого сообщения в сети полностью соответствует разработанному прикладному протоколу. Подтверждением корректной работы уровня представления является перехваченный TCP-пакет, в теле которого четко прослеживается структура передаваемого JSON-сообщения. Детальный просмотр инкапсулированных данных пакета представлен на рисунке 6.4.

```
{
  "type": "CONNECT",
  "payload": "2,....."
}

{
  "type": "BULLETS_UPDATE",
  "payload": "3"
}

{
  "type": "LOBBY_UPDATE",
  "payload": "..... (2/2)"
}

{
  "type": "LOBBY_UPDATE",
  "payload": ".....!"
}

{
  "type": "PLAYER_READY",
  "payload": "....."
}

{
  "type": "LOBBY_UPDATE",
  "payload": ".....!"
}

{
  "type": "LOBBY_UPDATE",
  "payload": ".....!"
}

{
  "type": "GAME_START",
  "payload": ".....!"
}

{
  "type": "CARDS_DEALT",
  "payload": "[
    {
      \"suit\": \"HEARTS\",
      \"rank\": \"JACK\"
    },
    {
      \"suit\": \"HEARTS\",
      \"rank\": \"SIX\"
    },
    {
      \"suit\": \"SPADES\",
      \"rank\": \"THREE\"
    },
    {
      \"suit\": \"HEARTS\",
      \"rank\": \"QUEEN\"
    },
    {
      \"suit\": \"SPADES\",
      \"rank\": \"KING\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"TEN\"
    },
    {
      \"suit\": \"DIAMONDS\",
      \"rank\": \"QUEEN\"
    },
    {
      \"suit\": \"SPADES\",
      \"rank\": \"JACK\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"TWO\"
    },
    {
      \"suit\": \"DIAMONDS\",
      \"rank\": \"EIGHT\"
    },
    {
      \"suit\": \"SPADES\",
      \"rank\": \"SEVEN\"
    },
    {
      \"suit\": \"SPADES\",
      \"rank\": \"QUEEN\"
    },
    {
      \"suit\": \"SPADES\",
      \"rank\": \"NINE\"
    },
    {
      \"suit\": \"DIAMONDS\",
      \"rank\": \"FOUR\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"FIVE\"
    },
    {
      \"suit\": \"DIAMONDS\",
      \"rank\": \"THREE\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"NINE\"
    },
    {
      \"suit\": \"DIAMONDS\",
      \"rank\": \"TEN\"
    },
    {
      \"suit\": \"HEARTS\",
      \"rank\": \"ACE\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"ACE\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"TWO\"
    },
    {
      \"suit\": \"DIAMONDS\",
      \"rank\": \"TWO\"
    },
    {
      \"suit\": \"HEARTS\",
      \"rank\": \"THREE\"
    },
    {
      \"suit\": \"HEARTS\",
      \"rank\": \"SEVEN\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"KING\"
    }
  ]"
}

{
  "type": "TABLE_UPDATE",
  "payload": "0;-;"
}

{
  "type": "LOBBY_UPDATE",
  "payload": "..... | .....:"
}

{
  "type": "YOUR_TURN",
  "payload": "....."
}

{
  "type": "PLAY_TURN",
  "payload": "[
    {
      \"suit\": \"HEARTS\",
      \"rank\": \"JACK\"
    },
    {
      \"suit\": \"HEARTS\",
      \"rank\": \"SIX\"
    },
    {
      \"suit\": \"SPADES\",
      \"rank\": \"THREE\"
    },
    {
      \"suit\": \"HEARTS\",
      \"rank\": \"QUEEN\"
    }
  ]"
}

{
  "type": "CARDS_DEALT",
  "payload": "[
    {
      \"suit\": \"SPADES\",
      \"rank\": \"KING\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"TEN\"
    },
    {
      \"suit\": \"DIAMONDS\",
      \"rank\": \"QUEEN\"
    },
    {
      \"suit\": \"SPADES\",
      \"rank\": \"JACK\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"TWO\"
    },
    {
      \"suit\": \"DIAMONDS\",
      \"rank\": \"EIGHT\"
    },
    {
      \"suit\": \"SPADES\",
      \"rank\": \"SEVEN\"
    },
    {
      \"suit\": \"SPADES\",
      \"rank\": \"QUEEN\"
    },
    {
      \"suit\": \"SPADES\",
      \"rank\": \"NINE\"
    },
    {
      \"suit\": \"DIAMONDS\",
      \"rank\": \"FOUR\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"FIVE\"
    },
    {
      \"suit\": \"DIAMONDS\",
      \"rank\": \"THREE\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"NINE\"
    },
    {
      \"suit\": \"DIAMONDS\",
      \"rank\": \"TEN\"
    },
    {
      \"suit\": \"HEARTS\",
      \"rank\": \"ACE\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"ACE\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"TWO\"
    },
    {
      \"suit\": \"DIAMONDS\",
      \"rank\": \"TWO\"
    },
    {
      \"suit\": \"HEARTS\",
      \"rank\": \"THREE\"
    },
    {
      \"suit\": \"HEARTS\",
      \"rank\": \"SEVEN\"
    },
    {
      \"suit\": \"CLUBS\",
      \"rank\": \"KING\"
    }
  ]"
}

{
  "type": "TABLE_UPDATE",
  "payload": "4;....."
}

{
  "type": "LOBBY_UPDATE",
  "payload": "..... 4 ..... | .....:"
}
```

Рисунок 6.4 – Передача сериализованного JSON-сообщения в процессе игры

Проведенное тестирование подтверждает, что серверная часть стабильно выдерживает подключения клиентов, игровая логика доводится до своего логического завершения, а обмен данными по сети происходит в строгом соответствии с заложенными протоколами и без потери информации.

7 ИНСТРУКЦИЯ ПОЛЬЗОВАТЕЛЯ И ТРЕБОВАНИЯ К ПРОГРАММНО-АППАРАТНОМУ ОБЕСПЕЧЕНИЮ

7.1 Требования к программно-аппаратному обеспечению

Для стабильного функционирования серверного узла и клиентских рабочих станций приложения «Карточный блеф» необходимо выполнение следующих минимальных системных требований:

- наличие установленной среды выполнения Java для запуска скомпилированных классов;
- поддержка операционной системой графической библиотеки Swing для корректной отрисовки пользовательского интерфейса;
- поддержка стека сетевых протоколов TCP/IP;
- доступность сетевого порта (по умолчанию используется порт 8080) для маршрутизации трафика к серверу;
- наличие стандартных устройств ввода (клавиатура и мышь) для взаимодействия с визуальными компонентами приложения;
- операционная система семейства Windows, Linux или macOS.

7.2 Инструкция пользователя

7.2.1 Подключение к серверу и подготовка к игре

Для начала работы пользователю необходимо запустить исполняемый файл клиентского приложения (GameClient). При запуске открывается главное окно с панелью конфигурации подключения, расположенной в верхней части экрана.

Процесс авторизации состоит из следующих шагов:

1. В поле «IP сервера» введите адрес компьютера, который выполняет роль сервера. Если вы играете с друзьями на одном устройстве или тестируете игру, оставьте значение по умолчанию (127.0.0.1).
2. В выпадающем списке «Игроков:» следует выбрать желаемое количество участников для предстоящей партии (допускается выбор 2, 3, 4 игроков).
3. В поле «Ник:» пользователь может задать свое игровое имя. Если поле оставить пустым, система автоматически сгенерирует имя вида «Player».
4. После заполнения данных необходимо нажать кнопку «Подключиться». Элементы ввода заблокируются, а кнопка «Я готов!» станет активной.
5. Для подтверждения своего участия в раздаче карт необходимо нажать кнопку «Я готов!». Интерфейс панели подключения представлен на рисунке 7.1.

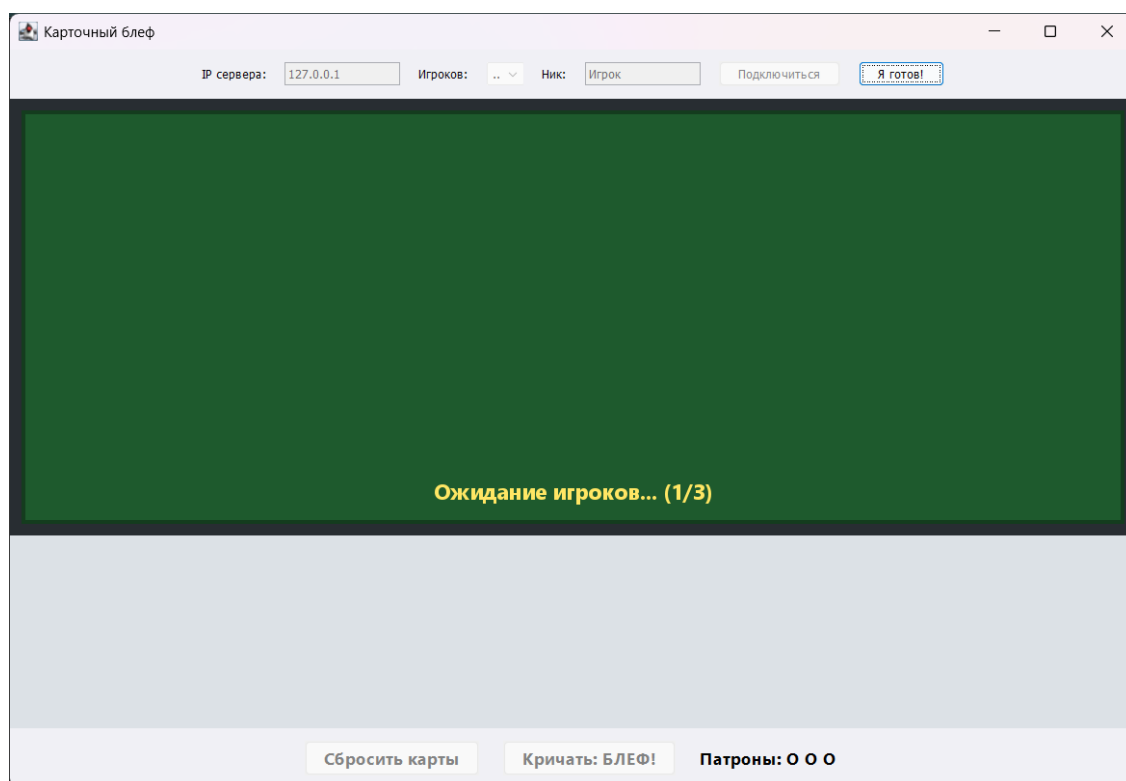


Рисунок 7.1 – Интерфейс настройки подключения и подтверждения готовности

7.2.2 Игровой процесс и элементы управления

После подтверждения готовности всеми участниками, сервер автоматически раздает карты и переводит интерфейс в игровой режим. Рабочая область приложения разделена на информационное табло (зеленое сукно), панель карт игрока (руку) и панель кнопок действий.

На информационном табло отображаются системные сообщения, текущий требуемый номинал карт («Требуется положить: ...») и номинал, заявленный предыдущим игроком («Последними выложили: ...»).

Для совершения хода пользователь должен выполнить следующие действия:

1. Дождаться своей очереди (на экране появится сообщение «ВАШ ХОД!»).
2. Кликнуть левой кнопкой мыши по картам на панели руки. Выбранные карты визуально сместятся вверх. Правилами допускается выбор от 1 до 4 карт одновременно.
3. Нажать кнопку «Сбросить карты». Карты будут отправлены на сервер и перенесены в общую стопку на столе. Визуализация процесса подготовки к ходу показана на рисунке 7.2.

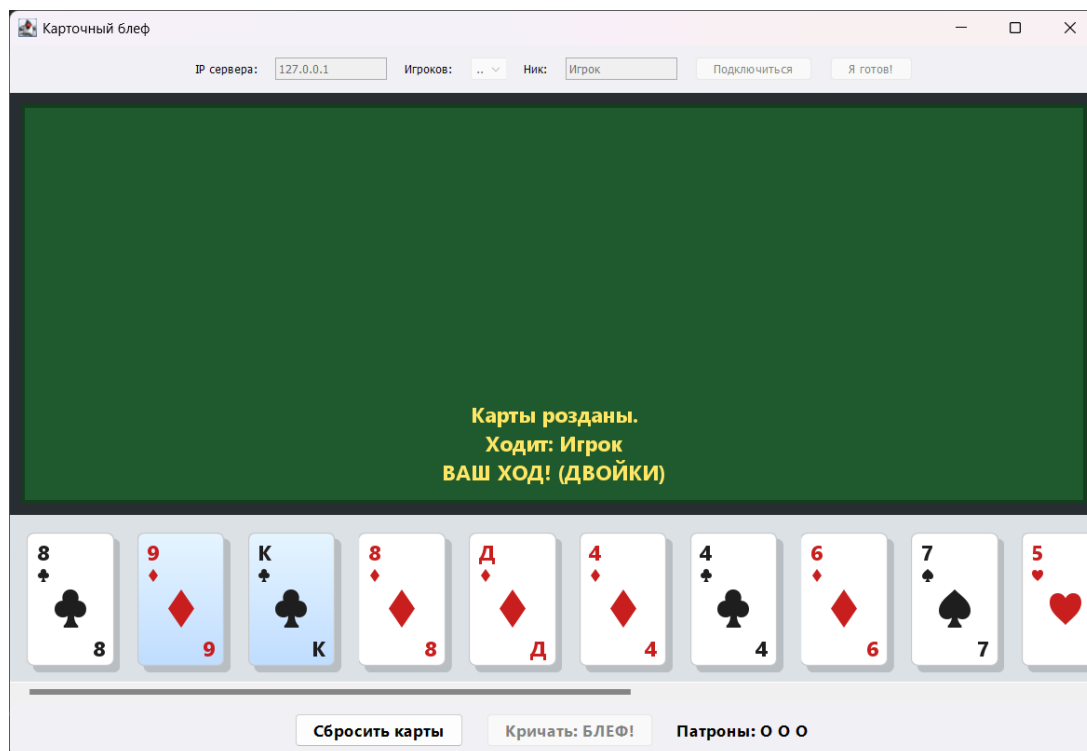


Рисунок 7.2 – Процесс выбора карт пользователем для совершения хода

7.2.3 Механика проверки на блеф

Вместо сброса собственных карт, активный игрок имеет право усомниться в честности предыдущего участника. Для этого предназначена кнопка «Кричать: БЛЕФ!» (выделена красным цветом).

Использование данной функции строго лимитировано механикой «патронов». Состояние счетчика отображается в нижнем правом углу экрана («Патроны: О О О»).

- В случае успешного выявления обмана, потраченный патрон возвращается игроку, а обманщик забирает все карты со стола.

- В случае ошибочного обвинения один патрон безвозвратно сгорает, а штрафные карты забирает обвинитель.

- Если количество патронов опускается до нуля (отображается статус «Патроны: ПУСТО»), кнопка проверки блокируется до конца игры.

Отображение состояния игровых ресурсов и панели действий представлено на рисунке 7.3.

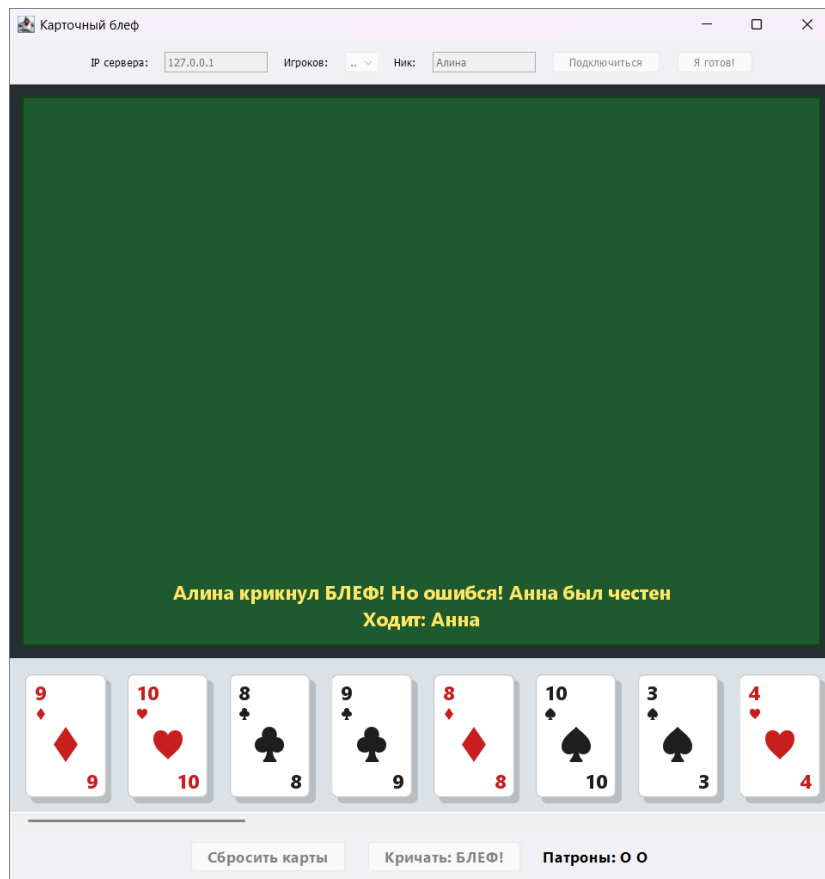


Рисунок 7.3 – Панель игровых действий и индикация доступных попыток проверок

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была успешно достигнута поставленная цель и разработано сетевое клиент-серверное приложение, реализующее многопользовательскую карточную игру «Карточный блеф» для 2–4 игроков. Изучены особенности предметной области сетевых многопользовательских карточных игр и проанализированы существующие аналоги, что позволило обосновать концепцию создания легковесного desktop-приложения. Обоснован выбор технологического стека, включающего язык программирования Java для обеспечения кроссплатформенности и стандартную графическую библиотеку Swing для построения визуальных компонентов.

В процессе работы спроектирована архитектура клиент-серверного взаимодействия по принципу «толстый» сервер и «тонкий» клиент, базирующаяся на прикладном, уровне представления, сеансовом и транспортном уровнях эталонной модели OSI. Разработана и реализована система сетевого обмена данными по надежному протоколу TCP с использованием собственного кастомного протокола прикладного уровня и JSON-сериализации текстовых сообщений. Сформулированы и программно реализованы ключевые алгоритмы игрового процесса, включающие систему очередей матчмейкинга, генерацию и тасование колоды из 52 карт, раздачу карт, алгоритм циклического перехода хода и обработку механики проверки условий «верю/не верю» с учетом лимита доступных «патронов».

Для удобства взаимодействия пользователей разработан интуитивно понятный графический интерфейс, обеспечивающий визуализацию игрового стола, автоматическое масштабирование и позиционирование карт на руках игрока, а также синхронное отображение динамических игровых событий. Испытана работоспособность созданного программного комплекса методами функционального тестирования, подтвердившими корректность отработки конечных состояний игры и алгоритмов определения победителя. С помощью программного анализатора сетевого трафика Wireshark показано, что обмен пакетами между клиентом и сервером происходит корректно, согласно заложенной структуре JSON-сообщений. Разработанное программное средство полностью автоматизирует процесс игры, берет на себя роль беспристрастного дилера и гарантирует честность игрового процесса за счет централизованного хранения эталонного состояния партии на сервере. Задание на курсовое проектирование выполнено в полном объеме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Архитектура многопользовательских онлайн-игр [Электронный ресурс] / Habr. – Режим доступа: <https://habr.com/ru/articles/829050/>.
- [2] Java: Введение и особенности языка [Электронный ресурс] / Metanit. – Режим доступа: <https://metanit.com/java/tutorial/1.1.php>.
- [3] Создание графического интерфейса с помощью Java Swing [Электронный ресурс] / Progoschool. – Режим доступа: <https://progoschool.ru/java/java-swing/>.
- [4] Транспортный уровень: протокол TCP [Электронный ресурс] / Habr. – Режим доступа: <https://habr.com/ru/articles/878534/>.
- [5] Сетевое программирование в Java: сокет (Socket) [Электронный ресурс] / JavaRush. – Режим доступа: <https://javarush.com/groups/posts/654-klassih-socket-i-serversocket-ili-allo-server-tih-menja-slihshishjh>.
- [6] Работа с JSON [Электронный ресурс] / MDN Web Docs. – Режим доступа: <https://developer.mozilla.org/ru/docs/Learn/JavaScript/Objects/JSON>.
- [7] Сериализация и десериализация в Java с использованием Gson [Электронный ресурс] / Pro-prof. – Режим доступа: <https://pro-prof.com/forums/topic/работа-c-json-na-java-библиотека-gson>.
- [8] Клиент-серверная архитектура в деталях [Электронный ресурс] / Habr. – Режим доступа: <https://habr.com/ru/articles/495698/>.
- [9] Многопоточность в Java: ExecutorService [Электронный ресурс] / Habr. – Режим доступа: <https://habr.com/ru/articles/116363/>.
- [10] Многопоточность (Multithreading) для практического программирования [Электронный ресурс] / Habr. – Режим доступа: <https://habr.com/ru/articles/761424/>.
- [11] Простое пособие по сетевой модели OSI для начинающих [Электронный ресурс] / Selectel. – Режим доступа: <https://selectel.ru/blog/osi-for-beginners/>.
- [12] Однопоточная модель Swing и Event Dispatch Thread [Электронный ресурс] / Javatutor. – Режим доступа: <https://javatutor.net/page/swing-events-and-threads.html/>.

ПРИЛОЖЕНИЕ А

Исходный код программы

```
Файл GameClient
package client;

import com.google.gson.Gson;
import shared.Message;
import shared.MessageType;

import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;
import java.util.ArrayList;
import java.util.List;

public class GameClient extends JFrame {
    private static final int SERVER_PORT = 8080;

    private JTextField serverIpField;
    private JComboBox<Integer> playersCountBox;
    private JTextField nicknameField;
    private JButton connectButton;
    private JButton readyButton;
    private JPanel handPanel;
    private TablePanel tableArea;
    private JButton playCardsButton;
    private JButton callBluffButton;
    private JLabel bulletsLabel;

    private final List<shared.Card> selectedCards = new ArrayList<>();
    private Socket socket;
    private PrintWriter out;
    private BufferedReader in;
    private final Gson gson = new Gson();

    private boolean isMyTurn = false;
    private String myUsername;

    public GameClient() {
        try {
            UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
        } catch (Exception ignored) {}

        setTitle("Карточный блеф");
        setSize(950, 650);
    }
}
```

```

setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setLocationRelativeTo(null);

JPanel topPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 10));
topPanel.setBackground(new Color(240, 240, 245));
topPanel.add(new JLabel("IP сервера:"));
serverIpField = new JTextField("127.0.0.1", 10);
topPanel.add(serverIpField);
topPanel.add(new JLabel("Игроков:"));
playersCountBox = new JComboBox<>(new Integer[]{2, 3, 4});
topPanel.add(playersCountBox);
topPanel.add(new JLabel("Ник:"));
nicknameField = new JTextField("Игрок", 10);
topPanel.add(nicknameField);
connectButton = new JButton("Подключиться");
readyButton = new JButton("Я готов!");
readyButton.setEnabled(false);
topPanel.add(connectButton);
topPanel.add(readyButton);

JPanel centerPanel = new JPanel(new BorderLayout(10, 0));
centerPanel.setBorder(new EmptyBorder(10, 10, 10, 10));
centerPanel.setBackground(new Color(40, 45, 50));

tableArea = new TablePanel();
tableArea.setBackground(new Color(30, 90, 45));
tableArea.setBorder(BorderFactory.createLineBorder(new Color(20, 60, 30), 4));
centerPanel.add(tableArea, BorderLayout.CENTER);

JPanel bottomPanel = new JPanel(new BorderLayout());
bottomPanel.setBackground(new Color(240, 240, 245));

handPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 15));
handPanel.setBackground(new Color(220, 225, 230));
JScrollPane handScroll = new JScrollPane(handPanel);
handScroll.setPreferredSize(new Dimension(950, 160));
handScroll.setBorder(null);
handScroll.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_NEVER);

JPanel actionPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 20, 10));
actionPanel.setBackground(new Color(240, 240, 245));
playCardsButton = new JButton("Сбросить карты");
playCardsButton.setFont(new Font("Segoe UI", Font.BOLD, 14));
callBluffButton = new JButton("Кричать: БЛЕФ!");
callBluffButton.setFont(new Font("Segoe UI", Font.BOLD, 14));
callBluffButton.setForeground(new Color(180, 0, 0));
playCardsButton.setEnabled(false);
callBluffButton.setEnabled(false);
bulletsLabel = new JLabel("Патроны: О О О ");
bulletsLabel.setFont(new Font("Segoe UI", Font.BOLD, 14));

```

```

        actionPanel.add(playCardsButton);
        actionPanel.add(callBluffButton);
        actionPanel.add(bulletsLabel);

        bottomPanel.add(handScroll, BorderLayout.CENTER);
        bottomPanel.add(actionPanel, BorderLayout.SOUTH);

        setLayout(new BorderLayout());
        add(topPanel, BorderLayout.NORTH);
        add(centerPanel, BorderLayout.CENTER);
        add(bottomPanel, BorderLayout.SOUTH);

        setupListeners();
    }

    private void setupListeners() {
        connectButton.addActionListener(e -> connectToServer());
        readyButton.addActionListener(e -> sendReadyStatus());

        playCardsButton.addActionListener(e -> {
            out.println(gson.toJson(new Message(MessageType.PLAY_TURN,
gson.toJson(selectedCards))));
            isMyTurn = false;
            updateActionButtons();
            selectedCards.clear();
        });

        callBluffButton.addActionListener(e -> {
            out.println(gson.toJson(new Message(MessageType.CALL_BLUFF, "Блеф!")));
            isMyTurn = false;
            updateActionButtons();
        });
    }

    private void updateActionButtons() {
        if (isMyTurn) {
            int totalHandCards = handPanel.getComponentCount();

            if (totalHandCards <= 1) {
                playCardsButton.setEnabled(selectedCards.size() == 1);
            } else {
                playCardsButton.setEnabled(selectedCards.size() >= 2 && selectedCards.size() <= 4);
            }
            callBluffButton.setEnabled(tableArea.getCardsCount() > 0);
        } else {
            playCardsButton.setEnabled(false);
            callBluffButton.setEnabled(false);
        }
    }

    private void connectToServer() {

```

```

try {
    String serverAddress = serverIpField.getText().trim();
    if (serverAddress.isEmpty()) return;

    int expectedPlayers = (Integer) playersCountBox.getSelectedItem();

    myUsername = nicknameField.getText().trim();
    if (myUsername.isEmpty()) {
        myUsername = "Player_" + new java.util.Random().nextInt(100);
        nicknameField.setText(myUsername);
    }

    socket = new Socket(serverAddress, SERVER_PORT);
    out = new PrintWriter(socket.getOutputStream(), true);
    in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

    connectButton.setEnabled(false);
    serverIpField.setEnabled(false);
    playersCountBox.setEnabled(false);
    nicknameField.setEnabled(false);
    readyButton.setEnabled(true);
    tableArea.setStatus("Подключение успешно. Ожидание...");

    String payload = expectedPlayers + "," + myUsername;
    out.println(gson.toJson(new Message(MessageType.CONNECT, payload)));
    new Thread(this::listenForMessages).start();
} catch (IOException ex) {
    JOptionPane.showMessageDialog(this, "Ошибка: " + ex.getMessage());
}

}

private void sendReadyStatus() {
    readyButton.setEnabled(false);
    out.println(gson.toJson(new Message(MessageType.PLAYER_READY, "Готов")));
}

private void resetToLobby() {
    try {
        if (socket != null && !socket.isClosed()) socket.close();
    } catch (IOException ignored) {}

    isMyTurn = false;
    tableArea.updateTable(0, "-", "-");
    tableArea.setStatus("Игра окончена. Можно начать заново.");
    handPanel.removeAll();
    handPanel.repaint();

    connectButton.setEnabled(true);
    serverIpField.setEnabled(true);
    playersCountBox.setEnabled(true);
    nicknameField.setEnabled(true);
}

```

```

readyButton.setEnabled(false);
playCardsButton.setEnabled(false);
callBluffButton.setEnabled(false);
bulletsLabel.setText("Патроны: О О О ");
}

private void listenForMessages() {
    try {
        String jsonLine;
        while ((jsonLine = in.readLine()) != null) {
            Message message = gson.fromJson(jsonLine, Message.class);

            switch (message.getType()) {
                case CARDS_DEALT -> {
                    java.lang.reflect.Type listType = new
com.google.gson.reflect.TypeToken<java.util.List<shared.Card>>(){}.getType();
                    java.util.List<shared.Card> hand = gson.fromJson(message.getPayload(),
listType);
                    SwingUtilities.invokeLater() -> renderHand(hand));
                }
                case TABLE_UPDATE -> {
                    String[] parts = message.getPayload().split(";");
                    int cardsOnTable = Integer.parseInt(parts[0]);
                    String lastRank = parts.length > 1 ? parts[1] : "-";
                    String targetRank = parts.length > 2 ? parts[2] : "-";
                    SwingUtilities.invokeLater() -> {
                        tableArea.updateTable(cardsOnTable, lastRank, targetRank);
                        updateActionButtons();
                    };
                }
                case YOUR_TURN -> SwingUtilities.invokeLater() -> {
                    isMyTurn = true;
                    updateActionButtons();
                    // Добавляем сообщение, используя разделитель " | " для авто-переноса
                    tableArea.setStatus(tableArea.getStatusMsg() + " | ВАШ ХОД! (" +
message.getPayload().toUpperCase() + ")");
                };
                case GAME_OVER -> {
                    String winner = message.getPayload();
                    SwingUtilities.invokeLater() -> {
                        if (winner.equals(myUsername)) {
                            JOptionPane.showMessageDialog(this, "🏆 Поздравляем! Вы победили!",
"Конец игры", JOptionPane.INFORMATION_MESSAGE);
                        } else {
                            JOptionPane.showMessageDialog(this, "💀 Вы проиграли. Победитель: "
+ winner, "Конец игры", JOptionPane.ERROR_MESSAGE);
                        }
                    };
                    resetToLobby();
                };
                break;
            }
        }
    }
}

```

```

        case BULLETS_UPDATE -> {
            int count = Integer.parseInt(message.getPayload());
            String bulletsText = "Патроны: " + "О ".repeat(Math.max(0, count));
            if (count == 0) bulletsText = "Патроны: ПУСТО";
            String finalBulletsText = bulletsText;
            SwingUtilities.invokeLater(() -> bulletsLabel.setText(finalBulletsText));
        }
        case LOBBY_UPDATE -> SwingUtilities.invokeLater(() -> {
            tableArea.setStatus(message.getPayload());
        });
        default -> {}
    }
}
} catch (IOException e) {
    SwingUtilities.invokeLater(() -> {
        if (!connectButton.isEnabled()) {
            tableArea.setStatus("Связь с сервером прервана.");
            resetToLobby();
        }
    });
}
}

private void renderHand(List<shared.Card> hand) {
    handPanel.removeAll();
    selectedCards.clear();
    for (shared.Card card : hand) {
        String suitSymbol = switch (card.suit()) {
            case HEARTS -> "♥"; case DIAMONDS -> "♦"; case CLUBS -> "♣"; case SPADES ->
"♠";
        };
        String rankStr = switch (card.rank()) {
            case TWO -> "2"; case THREE -> "3"; case FOUR -> "4"; case FIVE -> "5";
            case SIX -> "6"; case SEVEN -> "7"; case EIGHT -> "8"; case NINE -> "9"; case TEN -
> "10";
            case JACK -> "B"; case QUEEN -> "Д"; case KING -> "K"; case ACE -> "T";
        };
        Color cardColor = (card.suit() == shared.Suit.HEARTS || card.suit() ==
shared.Suit.DIAMONDS) ? new Color(200, 30, 30) : new Color(30, 30, 30);

        CardButton cardButton = new CardButton(rankStr, suitSymbol, cardColor);
        cardButton.addActionListener(e -> {
            if (selectedCards.contains(card)) {
                selectedCards.remove(card);
                cardButton.setSelectedState(false);
            } else {
                if (selectedCards.size() >= 4) return;
                selectedCards.add(card);
                cardButton.setSelectedState(true);
            }
        });
        updateActionButtons();
    }
}

```

```

        });
        handPanel.add(cardButton);
    }
    handPanel.revalidate();
    handPanel.repaint();
    updateActionButtons();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new GameClient().setVisible(true));
}

class TablePanel extends JPanel {
    private int cardsCount = 0;
    private String lastDeclared = "-";
    private String target = "-";
    private String statusMsg = "Добро пожаловать в игру!";

    public void updateTable(int count, String last, String nextTarget) {
        this.cardsCount = count;
        this.lastDeclared = last;
        this.target = nextTarget;
        repaint();
    }

    public void setStatus(String msg) {
        this.statusMsg = msg;
        repaint();
    }

    public String getStatusMsg() {
        return this.statusMsg;
    }

    public int getCardsCount() {
        return cardsCount;
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        g2.setColor(Color.WHITE);
        g2.setFont(new Font("Segoe UI", Font.BOLD, 18));

        if (!target.equals("-")) {
            g2.drawString("Требуется положить: " + target, 15, 30);
        }
    }
}

```

```

if (cardsCount > 0 && !lastDeclared.equals("-")) {
    g2.drawString("Последними выложили: " + lastDeclared, 15, 55);
}

// ИЗМЕНЕНИЯ ЗДЕСЬ: Логика отрисовки многострочного текста
if (!statusMsg.isEmpty()) {
    g2.setColor(new Color(255, 230, 100));
    g2.setFont(new Font("Segoe UI", Font.BOLD, 18)); // Немного уменьшен шрифт
    FontMetrics fm = g2.getFontMetrics();

    // Разбиваем сообщение по разделителю " | " (если он есть)
    String[] lines = statusMsg.split(" \\| ");

    // Вычисляем координату Y, чтобы последняя строка находилась у нижнего края
    (getHeight() - 25)
    int lineHeight = 25;
    int startY = getHeight() - 20 - (lines.length - 1) * lineHeight;

    for (String line : lines) {
        String cleanLine = line.trim();
        g2.drawString(cleanLine, (getWidth() - fm.stringWidth(cleanLine)) / 2, startY);
        startY += lineHeight;
    }
}

if (cardsCount == 0) {
    return;
}

int startX = getWidth() / 2 - 40;
int startY = getHeight() / 2 - 60;

for (int i = 0; i < cardsCount; i++) {
    double angle = Math.toRadians(-15 + (i * 7 % 30));
    int offsetX = i * 4;
    int offsetY = i * 2;

    g2.translate(startX + offsetX + 40, startY + offsetY + 60);
    g2.rotate(angle);

    g2.setColor(new Color(0, 0, 0, 60));
    g2.fillRoundRect(-37, -57, 80, 120, 10, 10);

    g2.setColor(new Color(25, 60, 120));
    g2.fillRoundRect(-40, -60, 80, 120, 10, 10);
    g2.setColor(Color.WHITE);
    g2.drawRoundRect(-40, -60, 80, 120, 10, 10);

    g2.setColor(new Color(255, 255, 255, 40));
    g2.fillOval(-20, -20, 40, 40);

```



```

        g2.rotate(-angle);
        g2.translate(-(startX + offsetX + 40), -(startY + offsetY + 60));
    }
}
}

class CardButton extends JButton {
    private final String rank;
    private final String suit;
    private final Color color;
    private boolean isSelectedState = false;

    public CardButton(String rank, String suit, Color color) {
        this.rank = rank;
        this.suit = suit;
        this.color = color;
        setPreferredSize(new Dimension(80, 120));
        setContentAreaFilled(false);
        setFocusPainted(false);
        setBorderPainted(false);
        setCursor(new Cursor(Cursor.HAND_CURSOR));
    }

    public void setSelectedState(boolean selected) {
        this.isSelectedState = selected;
        repaint();
    }

    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

        g2.setColor(new Color(0, 0, 0, 40));
        g2.fillRoundRect(5, 5, getWidth() - 5, getHeight() - 5, 12, 12);

        if (isSelectedState) {
            g2.setPaint(new GradientPaint(0, 0, new Color(230, 245, 255), 0, getHeight(), new
Color(190, 220, 255)));
        } else {
            g2.setColor(Color.WHITE);
        }
        g2.fillRoundRect(0, 0, getWidth() - 6, getHeight() - 6, 12, 12);

        g2.setColor(new Color(200, 200, 200));
        g2.drawRoundRect(0, 0, getWidth() - 6, getHeight() - 6, 12, 12);

        g2.setColor(color);
        g2.setFont(new Font("Segoe UI", Font.BOLD, 18));
        g2.drawString(rank, 8, 24);
    }
}

```

```

        g2.setFont(new Font("Segoe UI", Font.PLAIN, 16));
        g2.drawString(suit, 8, 42);

        g2.setFont(new Font("Segoe UI", Font.PLAIN, 46));
        FontMetrics fm = g2.getFontMetrics();
        int suitX = (getWidth() - 6 - fm.stringWidth(suit)) / 2;
        int suitY = (getHeight() - 6 - fm.getHeight()) / 2 + fm.getAscent() + 5;
        g2.drawString(suit, suitX, suitY);

        g2.setFont(new Font("Segoe UI", Font.BOLD, 18));
        fm = g2.getFontMetrics();
        g2.drawString(rank, getWidth() - 6 - fm.stringWidth(rank) - 8, getHeight() - 6 - 8);
    }
}
}

```

Файл ClientHandler
package server;

```

import com.google.gson.Gson;
import shared.Card;
import shared.Message;
import shared.MessageType;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;
import java.util.ArrayList;
import java.util.List;

public class ClientHandler implements Runnable {
    private final Socket socket;
    private final GameServer server;
    private GameSession session;

    private PrintWriter out;
    private BufferedReader in;
    private final Gson gson = new Gson();

    private boolean isReady = false;
    private String username;
    private List<Card> hand = new ArrayList<>();
    private int bullets = 3;

    public ClientHandler(Socket socket, GameServer server) {
        this.socket = socket;
        this.server = server;
    }

    public void setSession(GameSession session) { this.session = session; }
}

```

```

public boolean isReady() { return isReady; }
public String getUsername() { return username; }
public List<Card> getHand() { return hand; }
public void setHand(List<Card> hand) { this.hand = hand; }

public int getBullets() { return bullets; }
public void useBullet() { bullets--; sendBulletsCount(); }
public void addBullet() { bullets++; sendBulletsCount(); }

public void sendBulletsCount() {
    sendMessage(new Message(MessageType.BULLETS_UPDATE, String.valueOf(bullets)));
}

public void sendHand() {
    sendMessage(new Message(MessageType.CARDS_DEALT, gson.toJson(hand)));
}

@Override
public void run() {
    try {
        in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        out = new PrintWriter(socket.getOutputStream(), true);

        String jsonLine;
        while ((jsonLine = in.readLine()) != null) {
            Message message = gson.fromJson(jsonLine, Message.class);

            switch (message.getType()) {
                case CONNECT -> {
                    String[] parts = message.getPayload().split(",");
                    int count = Integer.parseInt(parts[0]);
                    this.username = parts[1];

                    sendBulletsCount();
                    server.addToQueue(this, count);
                }
                case PLAYER_READY -> {
                    this.isReady = true;
                    if (session != null) {
                        session.broadcast(new Message(MessageType.LOBBY_UPDATE,
this.username + " подтвердил готовность!"));
                        session.checkReadiness();
                    }
                }
                case PLAY_TURN -> {
                    java.lang.reflect.Type listType = new
com.google.gson.reflect.TypeToken<java.util.List<shared.Card>>(){}.getType();
                    List<shared.Card> playedCards = gson.fromJson(message.getPayload(),
listType);
                    if (session != null) session.handlePlayTurn(this, playedCards);
                }
            }
        }
    }
}

```

```

        }
        case CALL_BLUFF -> {
            if (session != null) session.handleCallBluff(this);
        }
        default -> {}
    }
}
} catch (IOException e) {
    System.out.println("Клиент " + username + " отключился.");
} finally {
    // Обязательно убираем игрока из очередей при разрыве соединения
    server.removeFromQueue(this);
}
}
}

public void sendMessage(Message message) {
    if (out != null) {
        out.println(gson.toJson(message));
    }
}
}
}

```

Файл `GameServer`
 package `server`;

```

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.ArrayList;
import java.util.List;

public class GameServer {
    private static final int PORT = 8080;

    // Очереди матчмейкинга для разного количества игроков
    private final List<ClientHandler> queue2 = new ArrayList<>();
    private final List<ClientHandler> queue3 = new ArrayList<>();
    private final List<ClientHandler> queue4 = new ArrayList<>();

    public void start() {
        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Сервер запущен. Ожидание игроков...");
            while (true) {
                Socket clientSocket = serverSocket.accept();
                ClientHandler handler = new ClientHandler(clientSocket, this);
                new Thread(handler).start();
            }
        } catch (IOException e) {
            System.err.println("Ошибка сервера: " + e.getMessage());
        }
    }
}

```

```

public synchronized void addToQueue(ClientHandler player, int count) {
    List<ClientHandler> targetQueue = switch (count) {
        case 3 -> queue3;
        case 4 -> queue4;
        default -> queue2;
    };

    targetQueue.add(player);
    System.out.println("Игрок " + player.getUsername() + " ищет игру на " + count + ". В
очереди: " + targetQueue.size() + "/" + count);

    // Оповещаем всех в очереди о текущем прогрессе сбора
    updateQueueStatus(targetQueue, count);

    // Если нужное количество набралось — создаем сессию
    if (targetQueue.size() == count) {
        List<ClientHandler> sessionPlayers = new ArrayList<>(targetQueue);
        targetQueue.clear(); // Очищаем очередь для следующих желающих

        GameSession session = new GameSession(sessionPlayers);
        for (ClientHandler p : sessionPlayers) {
            p.setSession(session);
        }
        // Запускаем проверку готовности
        session.initSession();
        System.out.println("Создана новая игровая сессия на " + count + " игроков.");
    }
}

// Удаляем игрока из очередей, если он отключился до старта
public synchronized void removeFromQueue(ClientHandler player) {
    if (queue2.remove(player)) updateQueueStatus(queue2, 2);
    if (queue3.remove(player)) updateQueueStatus(queue3, 3);
    if (queue4.remove(player)) updateQueueStatus(queue4, 4);
}

private void updateQueueStatus(List<ClientHandler> queue, int max) {
    for (ClientHandler p : queue) {
        p.sendMessage(new shared.Message(shared.MessageType.LOBBY_UPDATE,
"Ожидание игроков... (" + queue.size() + "/" + max + ")"));
    }
}

public static void main(String[] args) {
    new GameServer().start();
}
}

```

Файл GameSession
package server;

```

import shared.Card;
import shared.Message;
import shared.MessageType;
import shared.Rank;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public class GameSession {
    private final List<ClientHandler> players;
    private boolean gameStarted = false;
    private int currentPlayerIndex = -1;

    private final List<Card> tablePile = new ArrayList<>();
    private final List<Card> lastPlayedCards = new ArrayList<>();
    private ClientHandler lastPlayer;
    private Rank currentTargetRank = Rank.TWO;
    private Rank lastDeclaredRank = Rank.TWO;

    private String lastActionMessage = "";

    public GameSession(List<ClientHandler> players) {
        this.players = players;
    }

    public synchronized void broadcast(Message message) {
        for (ClientHandler player : players) player.sendMessage(message);
    }

    public void initSession() {
        broadcast(new Message(MessageType.LOBBY_UPDATE, "Комната собрана! Ждем готовности всех игроков..."));
        checkReadiness();
    }

    public synchronized void checkReadiness() {
        if (gameStarted) return;

        for (ClientHandler player : players) {
            if (!player.isReady()) return;
        }

        gameStarted = true;
        broadcast(new Message(MessageType.GAME_START, "Все готовы! Раздаем карты..."));
        dealCards();
    }

    private void dealCards() {
        Deck deck = new Deck();
        deck.shuffle();
    }
}

```

```

int cardsPerPlayer = deck.cardsLeft() / players.size();

for (ClientHandler player : players) {
    List<Card> hand = new ArrayList<>();
    for (int i = 0; i < cardsPerPlayer; i++) hand.add(deck.drawCard());
    player.setHand(hand);
    player.sendHand();
}
broadcast(new Message(MessageType.TABLE_UPDATE, "0;-;-"));
lastActionMessage = "Карты розданы.";
startFirstTurn();
}

private void startFirstTurn() {
    currentPlayerIndex = new Random().nextInt(players.size());
    currentTargetRank = Rank.TWO;
    nextTurn();
}

public synchronized void nextTurn() {
    ClientHandler activePlayer = players.get(currentPlayerIndex);

    String turnMsg = "Ходит: " + activePlayer.getUsername();

    if (!lastActionMessage.isEmpty()) {
        // Используем чёткий разделитель " | " для удобного переноса строк на клиенте
        broadcast(new Message(MessageType.LOBBY_UPDATE, lastActionMessage + " | " +
turnMsg));
    } else {
        broadcast(new Message(MessageType.LOBBY_UPDATE, turnMsg));
    }

    String requiredRank = getRankNameInRussian(currentTargetRank);
    activePlayer.sendMessage(new Message(MessageType.YOUR_TURN, requiredRank));
}

public synchronized void handlePlayTurn(ClientHandler player, List<Card> playedCards) {
    if (players.indexOf(player) != currentPlayerIndex) return;

    lastPlayer = player;
    lastPlayedCards.clear();
    lastPlayedCards.addAll(playedCards);
    tablePile.addAll(playedCards);

    player.getHand().removeAll(playedCards);
    player.sendHand();

    lastDeclaredRank = currentTargetRank;
    int numCards = playedCards.size();

    if (player.getHand().isEmpty()) {

```

```

        broadcast(new Message(MessageType.TABLE_UPDATE, tablePile.size() + ";" +
getRankNameInRussian(lastDeclaredRank) + ";-"));
        broadcast(new Message(MessageType.LOBBY_UPDATE, "🏆 " + player.getUsername()
+ " ПОБЕДИЛ!"));
        broadcast(new Message(MessageType.GAME_OVER, player.getUsername()));
        return;
    }

    advanceTargetRank();

    broadcast(new Message(MessageType.TABLE_UPDATE, tablePile.size() + ";"
+ getRankNameInRussian(lastDeclaredRank) + ";"
+ getRankNameInRussian(currentTargetRank)));

    lastActionMessage = player.getUsername() + " положил " + numCards + " шт. как " +
getRankNameInRussian(lastDeclaredRank);

    currentPlayerIndex = (currentPlayerIndex + 1) % players.size();
    nextTurn();
}

public synchronized void handleCallBluff(ClientHandler caller) {
    if (lastPlayedCards.isEmpty()) {
        caller.sendMessage(new Message(MessageType.LOBBY_UPDATE, "На столе пусто!
Блефовать пока нельзя.));
        caller.sendMessage(new Message(MessageType.YOUR_TURN,
getRankNameInRussian(currentTargetRank)));
        return;
    }

    if (caller.getBullets() <= 0) {
        caller.sendMessage(new Message(MessageType.LOBBY_UPDATE, "У вас закончились
патроны!));
        caller.sendMessage(new Message(MessageType.YOUR_TURN,
getRankNameInRussian(currentTargetRank)));
        return;
    }

    caller.useBullet();

    boolean lied = false;
    for (Card c : lastPlayedCards) {
        if (c.rank() != lastDeclaredRank) {
            lied = true;
            break;
        }
    }

    if (lied) {
        // Обновленное сообщение при успешном блефе
        lastActionMessage = "🔥 " + caller.getUsername() + " крикнул БЛЕФ! И оказался

```



```

прав! " + lastPlayer.getUsername() + " забирает карты";
    lastPlayer.getHand().addAll(tablePile);
    lastPlayer.sendHand();
    caller.addBullet();
    caller.sendMessage(new Message(MessageType.LOBBY_UPDATE, "Вы были правы!
Патрон возвращен."));
    currentPlayerIndex = players.indexOf(caller);
} else {
    // Обновленное сообщение при провальном блефе
    lastActionMessage = " ❌ " + caller.getUsername() + " крикнул БЛЕФ! Но ошибся! " +
lastPlayer.getUsername() + " был честен";
    caller.getHand().addAll(tablePile);
    caller.sendHand();
    caller.sendMessage(new Message(MessageType.LOBBY_UPDATE, "Вы ошиблись.
Один патрон сгорел."));
    currentPlayerIndex = players.indexOf(lastPlayer);
}

    tablePile.clear();
    lastPlayedCards.clear();
    currentTargetRank = lastDeclaredRank;
    broadcast(new Message(MessageType.TABLE_UPDATE, "0;-;-"));
    nextTurn();
}

private void advanceTargetRank() {
    int nextOrdinal = (currentTargetRank.ordinal() + 1) % Rank.values().length;
    currentTargetRank = Rank.values()[nextOrdinal];
}

private String getRankNameInRussian(Rank rank) {
    return switch (rank) {
        case TWO -> "Двойки"; case THREE -> "Тройки"; case FOUR -> "Четверки";
        case FIVE -> "Пятерки"; case SIX -> "Шестерки"; case SEVEN -> "Семерки";
        case EIGHT -> "Восьмерки"; case NINE -> "Девятки"; case TEN -> "Десятки";
        case JACK -> "Валеты"; case QUEEN -> "Дамы"; case KING -> "Короли"; case ACE -
> "Тузы";
    };
}
}

```

